generic `404` page with very little information on it. You don't want to expose the details of your code to the untrustworthy public.

In a development environment, where `DEBUG=True`, Django shows you a debugging error page, complete with information on what went wrong. In the case of a bad `Musician` ID, the error is *No Musician matches the given query*. The error page also includes a list of all the URLs Django attempted to match. This can be handy if you're trying to figure out why the URL you typed is causing a `404` error. In the case of a `404` raised by `get_object_or_404()`, you'll see the URL that got matched, but that the argument wasn't valid. Figure 4.7 contains the debug error page for a musician ID that doesn't exist in my database.

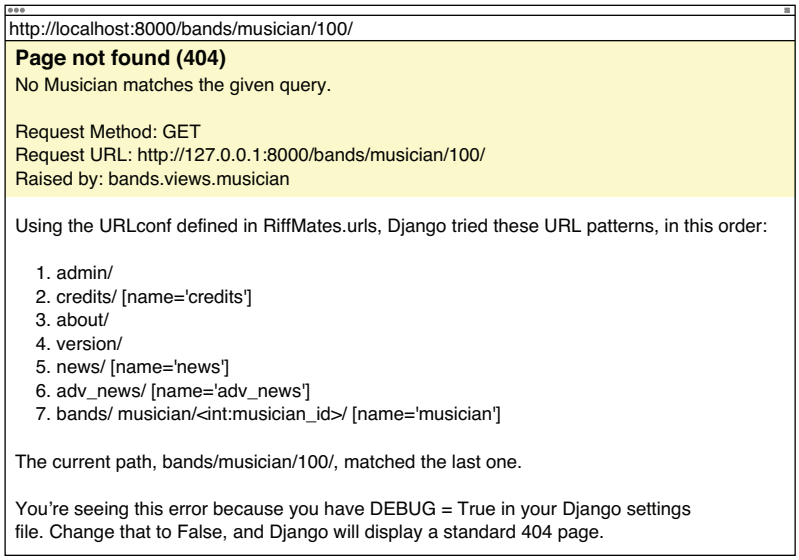


Figure 4.7 Page not found, as no musician exists with ID 100

Getting a 500 error

Pointing to a bad URL, or passing invalid arguments to one, results in a `404` page. Coding errors result in a 500 error. If `DEBUG=True`, the 500 error page is similar to the `404` and includes a stack trace of your code, showing the offending line and a variety of information about the calling state. Like with the `404`, in production mode, the 500 error is generic, showing very little information, and it also can be customized for your application.

The visitors to your site probably aren't interested in guessing the ID of the musician they're looking for. In addition to your musician's details page, you're going to want some sort of index listing as well.